Pre-training BERT with Permuted Language Model带读

PERT: Pre-training BERT with Permuted Language Model：用“位置预测”替代 MLM，探索更像“纠错读句子”的预训练任务

0. TL;DR（≤120字）

- 贡献一句话：提出 PerLM（Permuted Language Model）—把句子局部打乱，让模型预测原词位置而非预测词本身。
- 适用一句话：适合想增强“序列/顺序建模能力”的 NLU 编码器预训练或继续预训练（DAPT）。
- 风险一句话：对短文本分类可能负迁移；预训练用“乱序”，微调用“正常序”，存在任务失配。

---

1. 背景与动机：他们到底在“卡什么”

- 任务背景：PLM（Pre-trained Language Model，预训练语言模型）主流分两派：自回归（auto-regressive，如 GPT）与自编码（auto-encoding，如 BERT）。论文聚焦后者。
- 任务背景：自编码 PLM 的核心预训练任务几乎被 MLM（Masked Language Model）统治：把少量 token 替换成 `[MASK]`，再在词表空间预测被遮住的 token。
- 痛点拆解：MLM 很强，但它“强到有点单调”——大量工作只是 MLM 的变体（WWM、N-gram masking 等），创新空间被压得很窄。
- 痛点拆解：MLM 还带来经典的预训练-微调不一致（pretraining-finetuning discrepancy）：预训练时见过 `[MASK]`，下游推理时永远不会见到。
- 作者的关键洞见：人类读句子时，即使局部词序被打乱，也能依靠上下文恢复大意；那能不能把“恢复词序”当成一种预训练任务？（论文用中文乱序例子直观展示）
- 作者的关键洞见（一句话点题）：用 **PerLM（Permuted Language Model）** 把输入句子局部打乱，让模型 **预测“原词的位置”** 来学上下文表示，而不是预测词本身。

---

2. 问题定义与符号（足够让人复现）

2.1 输入/输出：清晰列出

- 输入：一段文本序列（可单句，也可像 BERT 那样是句对）。论文形式化为两段序列
  - $A = \{A_1, \ldots, A_n\}$，$B = \{B_1, \ldots, B_m\}$
- 预处理：对 A, B 做“局部置乱”，得到 A', B'。
- 拼接成最终输入（BERT 风格）：

$$X = [CLS]\ A'_1 \ldots A'_n\ [SEP]\ B'_1 \ldots B'_m\ [SEP]$$

- 输出（训练监督信号）：对被选中参与置乱的 token，输出它在 **原序列** 中的 **位置 index**（即“这个 token 原本在哪个位置”）。

2.2 目标函数/评价指标

核心：位置分类（position classification）而不是词表分类（vocab classification）。

- 编码器输出：

$$H^{(0)} = \text{Embedding}(X), \quad H^{(i)} = \text{Transformer}(H^{(i-1)}), \quad H = H^{(L)}$$

其中 $H \in \mathbb{R}^{N \times d}$。

- 取出被选中 token 的隐状态 $H_m \in \mathbb{R}^{k \times d}$（$k = \lfloor 0.15N \rfloor$，默认 15% 的词参与 masking/置乱）。
- 过一层 FFN + Dropout + LayerNorm 得到 $\tilde{H}_m$。
- 位置分布用“与全序列隐状态做点积”的方式得到（长度为 L 的分布，L≈序列长度）：

$$p_i = \text{softmax}(\tilde{H}_{m,i} H^\top + b) \in \mathbb{R}^L$$

- 交叉熵损失：

$$\mathcal{L} = -\frac{1}{M} \sum_{i=1}^{M} y_i \log p_i$$

其中 y_i 是“正确位置”的 one-hot 标签。

2.3 关键假设：作者默认了什么

- 句子局部乱序仍保留足够可恢复信息；模型能通过上下文与语法线索推断“正确位置”。
- 预测空间从“词表大小 $|V|$”转为“序列长度 L”，**计算成本可控**，且能减少对 `[MASK]` 的依赖。
- 预训练时允许乱序，但微调时用自然顺序；作者也承认这可能引入失配与“un-natural word orders”的副作用。

---

3. 方法总览：一张“脑内结构图”

3.1 总流水线（文字 + 列表 + ASCII）

```
│
├──(1) 选 token：WWM + N-gram masking（40/30/20/10 分配给 1~4gram）
│
├──(2) 置乱：对选中词的 90% 做 shuffle；剩余 10% 不动当负样本（negative samples）
│   （注意：只在这 90% 上 shuffle，不是全句乱序）
│
├──(3) 编码：BERT-style embedding + L 层 Transformer → H ∈ R^{N×d}
│
├──(4) 取子集：取被选中的 k 个位置 → Hm → FFN+Dropout+LayerNorm → ĤH_m
/
└──(5) 位置预测：p_i = softmax(ĤH{m,i} H^T + b)，交叉熵训练
```

3.2 每一步的“作用”和“替代方案”

1. 选 token（WWM/N-gram）

- 作用：让置乱发生在“词/短语”级别，迫使模型理解更大粒度的结构。
- 替代：纯 token-level 随机选；Span corruption（BART 风格）；随机窗口内置乱。

2. 置乱 + 负样本（shuffle + negative samples）

- 作用：让模型学“顺序修复”；负样本让模型知道“不是所有 token 都应该被移动/匹配”，避免退化为机械对齐。
- 替代：不设负样本；用随机词替换当负样本；全句置乱（更重）。

3. 编码器（BERT 结构）

- 作用：保留双向上下文（bidirectional context），对齐 BERT 的下游使用方式。
- 替代：Transformer-Encoder 任意实现；或者直接用 seq2seq 编码器作为 backbone。

4. 位置预测头（dot product vs. classifier）

- 作用：用点积把“候选 token 表示”与“全序列表示”做匹配，本质像一种“序列内检索/对齐”。
- 替代：MLP 分类到 $[0..L-1]$；或者用 Hungarian matching / Sinkhorn 做更硬的排列学习（更重）。

---

4. 核心方法逐段讲解（按论文逻辑走）

4.1 模块 A：PerLM 的数据构造（它做什么、怎么做、为何这样做）

直觉（为什么有效）

把“理解句子”转化为“把乱序句子纠回去”。这会逼模型利用：

- 局部搭配（collocation）
- 句法线索（syntactic cues）
- 长距离依赖（long-range dependency）
  来判断“这个词应该插在哪”。

形式化（关键规则）

- 选中 15% 的输入词作为候选（masking ratio = 15%）。
- 候选词的来源：WWM（Whole Word Masking）+ N-gram masking（按 40/30/20/10 分配给 1~4gram）。
- 对候选词中的 90%：打乱它们的顺序（shuffle）。
- 剩余 10%：保持原位，作为负样本（negative samples）。
  - 重要细节：shuffle **只发生在这 90% 的候选词集合内部**，不是全句乱序；这让难度处于“可学”范围。

伪代码（数据构造）

```
input: tokens t[0..N-1]

C = select_candidates_by_WWM_and_ngram(t, ratio=0.15, ngram_mix=[0.4,0.3,0.2,0.1])

S = random_subset(C, ratio=0.9)     # to shuffle
Neg = C \ S                         # keep unchanged as negative samples

perm = random_permutation(S)
t' = t.copy()
for idx, new_tok in zip(S, permuted_tokens_from(S)):
    t'[idx] = new_tok

labels = { idx → original_position_of(t'[idx]) } for idx in C
return t', labels
```

工程细节（实现坑、默认超参、复杂度）

- 坑 1：**“词”与 “token”** 的边界。作者在中文里用分词工具只为 WWM 选“整词”，最终输入仍是 WordPiece

token 序列。

- 坑 2：位置预测的标签必须对齐到 **原始序列的位置 index**，不是 shuffle 后的位置。
- 坑 3：如果你实现“全句乱序”，难度会飙升；作者实验表明全局置乱更强但更偏“极端设置”。
- 复杂度：位置分布是对长度 L 的 softmax，计算形态是 $k \times L$ 的相似度矩阵（点积），比 $k \times |V|$（MLM）便宜得多，尤其当词表很大时。

4.2 模块 B：位置预测头（为什么是点积 + softmax）

直觉

这其实是在做“序列内检索”：

- 给定一个被置乱的 token 表示 $\tilde{h}_i$，去全序列表示 H 里找“最像它原位置上下文的那个位置”。
  点积是最朴素、也最稳定的匹配算子。

形式化（核心公式）

$$p_i = \mathrm{softmax}(\tilde{H}_{m,i} H^\top + b)$$
$$\mathcal{L} = -\frac{1}{M} \sum_i y_i \log p_i$$

工程细节

- 训练稳定性：点积 logits 规模受 hidden norm 影响，FFN + LayerNorm 这套在实践里通常能稳住。
- 输出长度问题：如果你用动态 padding，要确保 L 是有效长度（或把 padding 位置 mask 掉），否则 softmax 会把概率浪费在 padding 上。

4.3 模块 C：只在被选位置预测（partial prediction vs. full prediction）

作者强调：像 MLM 一样，只对被选中的位置做预测即可，不需要对全序列每个 token 都预测。
在分析部分他们也做了对比：**partial prediction 比 full prediction 更好**（同等预算下更专注、更像 MLM 的学习信号），属于“复现时很容易忽略但很关键”的点。

---

5. 训练/推理细节（工程读者最关心的部分）

5.1 数据处理

- **中文预训练语料**：沿用 MacBERT 的 20G 级别中文语料（中文 Wiki、百科、社区问答、新闻等），约 5.4B words。
- **中文 tokenizer**：WordPiece；中文分词工具只用于 WWM 选词边界，不改变最终 token 序列。
- **英文预训练语料**：Wiki + BooksCorpus（经典配置）。
- 关键提醒：迁移到别的领域（比如 Deep Past）最常见坑是 **“WWM 的词边界怎么定义”** ——领域符号化文本往往没有现成分词器。

5.2 训练策略（超参）

论文给了比较完整的 recipe（中英类似）：

- max length = 512
- batch size：base 级别 416；large 级别 128
- lr = 1e-4，warmup 10k steps，total steps = 2M
- AdamW：weight decay 0.1，betas (0.9,0.999)，eps 1e-6

- 训练设备：单个 TPU v3-8（128G HBM）

> 隐含信息：这是“从零预训练”配置；我们 8GB 场景应走“继续预训练/多任务微调”的轻量路径（见第 8 节）。

5.3 推理策略

- PERT 本质是 BERT 类编码器：下游推理与 BERT 一样做分类/抽取/序列标注。
- 重要细节：**微调与推理阶段使用自然顺序输入**，不再乱序；这既是优点（兼容 BERT 脚本），也是风险（预训练/微调分布不一致）。

---

6. 实验设计：他们怎么证明“不是玄学”

6.1 数据集与划分（是否泄漏风险）

- 中文：MRC（CMRC2018、DRCD）、TC（XNLI、LCQMC、BQ、ChnSentiCorp、TNEWS、OCNLI）、NER（MSRA-NER、People’s Daily）。
- 英文：SQuAD 1.1/2.0、MNLI、SST-2、CoLA、MRPC。
- 公平性：只对比“同预训练语料”的模型版本；中文做 10 seeds 重复并报告 max/avg（英文 5-run avg）。

6.2 Baseline（对照组）

- 中文：BERT/RoBERTa/ELECTRA/MacBERT（base/large）。
- 英文：BERT、RoBERTa、XLNet、ALBERT 等（保证尽量同预训练数据）。

6.3 结果解读（趋势解释）

- (a) 中文 MRC：提升明显。长上下文 + 强顺序/对齐需求任务更受益。
- (b) 中文 NER：整体增益。序列标注对局部上下文与位置敏感。
- (c) 中文 TC：多数任务落后。短文本语义任务可能被乱序噪声抵消。
- (d) 英文：SQuAD 系列较强，但分类/语义等价任务不一定占优。

6.4 消融：最关键结论（工程优先级）

1. 置乱范围（window size）：全局置乱最好（更强但更极端）。
2. masking ratio：15% 最好。
3. shuffle ratio：90% 最好。
4. negative sample ratio：10% 最好。
5. partial prediction > full prediction：只预测被选位置更好。

6.5 失败案例/潜在失败类型（推断）

- 【推断】短文本/句对分类：结构偏置收益有限，可能负迁移。
- 【推断】paraphrase/语义等价：顺序修复不是主要瓶颈。
- 【推断】域外符号化文本：若顺序线索弱或格式奇特，增益不稳。

---

7. 批判性复盘：这篇论文哪些地方可能不靠谱

- 强依赖前提：文本必须有可学习的顺序规律；极端自由语序或强省略文本收益可能更小。
- 负迁移点：预训练乱序、微调正序的分布缺口；分类任务已显露风险。
- 复现坑：WWM/N-gram 实现细节；位置标签对齐；partial vs full prediction；padding mask。

- 边界：它是 encoder-only NLU 框架；生成式任务要迁移“思想”而非照搬结构。

---

8. 迁移到 Deep Past（Akkadian→English）的落地方案

8.1 我们的 pipeline 插入点

1. **Tokenizer / 数据层（最便宜）**

- 源端（transliteration）做轻量局部置乱增强（permutation augmentation）。
- 控制强度：别把语法/结构线索彻底打散。

2. **Model / Training 层（更值钱）**：加 **PerLM 辅助损失（auxiliary loss）**

$$\mathcal{L} = \mathcal{L}_{MT} + \lambda \mathcal{L}_{PerLM}$$

- 在 seq2seq 的 encoder 输出上加位置预测头，做多任务训练。

3. **Pretraining / DAPT（继续预训练）**

- 若合规可用无标注转写：先 PerLM 继续预训练，再用并行数据 fine-tune。
- 不引外部数据：只用官方源端做小规模自监督（低成本、收益不确定但可试）。

8.2 最小可复现实验（MVP）

对照组设置

- Exp0：标准 seq2seq 监督训练。
- Exp1：+源端局部置乱增强（选 5%~10% token，窗口≤4 shuffle）。
- Exp2：+Aux PerLM（PerLM batch 比例 25%，λ 从 0.1 起试）。

需要改动的代码/配置点（按模块列）

- Dataset：返回 `src_ids, tgt_ids, permuted_src_ids, position_labels, perm_mask`。
- Model：encoder 输出 `H (B×L×d)`；gather `H_m (B×k×d)`；`logits = Hm_tilde @ H.transpose(-1,-2) + b`（B×k×L）；CE loss（mask 掉 padding）。
- Trainer：混合 loss；PerLM batch 占比与 λ 可配置。

验收指标

- 主指标：BLEU、chrF++。
- 汇总：几何均值/加权平均（与比赛策略一致）。
- 额外诊断：长度比（hyp/ref），copy-rate，低频 token 覆盖率。

8.3 预计算力（8GB）与降级方案

- 不从零预训练；做轻量继续预训练/多任务微调。
- 建议：fp16/bf16、gradient accumulation、gradient checkpointing、LoRA/QLoRA、先 256 再 512。

8.4 进一步迭代路线（≤5 条，按优先级）

1. 置乱超参网格：mask(5/10/15)、window(2/4/8)、shuffle(50/90)。
2. 位置标签改为相对/局部位置，减少结构破坏。
3. 两阶段：先 PerLM N steps，再 MT fine-tune，对比多任务。
4. 叠加约束解码（术语表/专名对齐）—后期再做。
5. 错误分析驱动增广：专盯数字/专名/表格结构/公式样式。

9. 读完该做什么（行动清单）

立刻能做的 3 件事（1 小时内）

1. 统计 Deep Past 源端 tokenization：平均长度、长尾 token、词边界是否天然存在。
2. 写最小“局部置乱增强”函数（window shuffle），先跑 Exp1。
3. sanity check：抽 50 条看增强是否把结构彻底毁掉。

需要半天的 3 件事

1. 在 seq2seq 代码里加 PerLM 辅助头（先能算 loss 即可）。
2. 跑 Exp0/Exp1/Exp2 对照，固定 seed 至少 3 次看方差。
3. 加诊断指标（长度比、copy-rate），避免单指标误导。

需要 1~3 天验证的 3 件事

1. 系统搜索置乱超参，找“稳健小增益”区域。
2. 对比两阶段 DAPT vs 多任务方案。
3. 聚类失败样本（专名、数字、句长、结构），定位 PerLM 增益/伤害区间。